

EXCEPTIONS

Dr. Subhi A. Rahim
Bahudaila

Lecture 7

OOP

WHY DO WE NEED EXCEPTIONS?

2 Lecture 7

Exceptions are errors that occur at runtime.

- C-language programs often signal an error by returning a particular value from the function in which it occurred.
- The problem becomes more complex when classes are used, since errors may take place without a function being explicitly called. For example, an application defines objects of a class: `SomeClass obj1, obj2, obj3;`

How will the application find out if an error occurred in the class constructor? The constructor is called implicitly, so there's no return value to be checked.

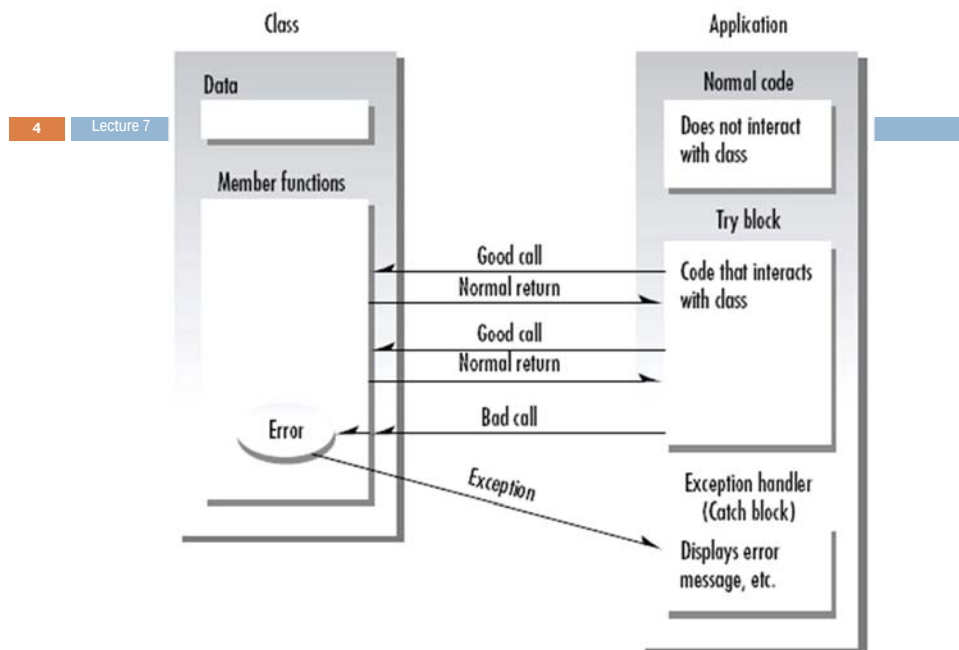
- Exception handling provides a standard mechanism for processing errors. This is especially important when working on a project with a large team of programmers.

EXCEPTION SYNTAX

3 Lecture 7

When an error to be detected in a member function. This member function then informs the application that an error has occurred. When exceptions are used, this is called **throwing an exception**. In the application we install a separate section of code to handle the error. This code is called **an exception handler or catch block**; it catches the exceptions thrown by the member function. Any code in the application that uses objects of the class is enclosed in a **try block**. Errors generated in the try block will be caught in the **catch block**.

Code that doesn't interact with the class need not be in a try block. The figure here shows the arrangement.



The exception mechanism

A SIMPLE EXCEPTION EXAMPLE

5

Lecture 7

The application program might attempt to push too many objects onto the stack, thus exceeding the capacity of the array, or it might try to pop too many objects off the stack, thus obtaining invalid data. In the XSTAK program we use an exception to handle these two errors.

```
// xstak.cpp
// demonstrates exceptions
#include <iostream>
using namespace std;
const int MAX = 3; //stack holds 3 integers
///////////////////////////////////////////////////
6 class Stack
{
private:
    int st[MAX]; //array of integers
    int top; //index of top of stack
public:
    class Range //exception class for Stack
    { //note: empty class body
    };
    Stack() //constructor
    { top = -1; }
    void push(int var)
    {
        if(top >= MAX-1) //if stack full,
            throw Range(); //throw exception
        st[++top] = var; //put number on stack
    }
    int pop()
    {
        if(top < 0) //if stack empty,
            throw Range(); //throw exception
        return st[top--]; //take number off stack
    }
};
///////////////////////////////////////////////////
```

Specifying the Exception

Class: the class name Range is used to connect a throw statement with a catch block.

Throwing an Exception: throw Range();

The Range() part of this statement invokes the implicit constructor for the Range class, which creates an object of this class. The throw part of the statement transfers program control to the exception handler catch(Stack::Range).

The try Block: All the statements in main() that might cause this exception are allocated in the try block.

7

Lecture 7

```

int main()
{
    Stack s1;
    try
    {
        s1.push(11);
        s1.push(22);
        s1.push(33);
        // s1.push(44); //oops: stack full
        cout << "1: " << s1.pop() << endl;
        cout << "2: " << s1.pop() << endl;
        cout << "3: " << s1.pop() << endl;
        cout << "4: " << s1.pop() << endl; //oops: stack empty
    }
    catch(Stack::Range) //exception handler
    {
        cout << "Exception: Stack Full or Empty" << endl;
    }
    cout << "Arrive here after catch (or normal exit)" << endl;
    return 0;
}

```

1: 33

2: 22

3: 11

Exception: Stack Full or Empty

Arrive here after catch (or normal exit)

The output

MULTIPLE EXCEPTIONS

8

Lecture 7

You can design a class to throw as many exceptions as you want, by modifying the XSTAK program to throw separate exceptions for attempting to push data on a full stack and attempting to pop data from an empty stack.

```

// xstak2.cpp
// demonstrates two exception handlers
#include <iostream>
using namespace std;
const int MAX = 3; //stack holds 3 integers
////////////////////////////////////
class Stack
{
private:
    int st[MAX]; //stack: array of integers
    int top; //index of top of stack
public:
    class Full { }; //exception class
    class Empty { }; //exception class
    //-----
    Stack() //constructor
    { top = -1; }
    //-----
    void push(int var) //put number on stack
    {
        if(top >= MAX-1) //if stack full.
            throw Full(); //throw Full exception
        st[++top] = var;
    }
    //-----
    int pop() //take number off stack
    {
        if(top < 0) //if stack empty.
            throw Empty(); //throw Empty exception
        return st[top--];
    }
};
////////////////////////////////////

```

The statement **throw Full();** is executed if the application calls push() when the stack is already full, and **throw Empty();** is executed if pop() is called when the stack is empty. A separate catch block is used for each exception:

10 Lecture 7

```

int main()
{
    Stack s1;
    try
    {
        s1.push(11);
        s1.push(22);
        s1.push(33);
        //s1.push(44); //oops: stack full
        cout << "1: " << s1.pop() << endl;
        cout << "2: " << s1.pop() << endl;
        cout << "3: " << s1.pop() << endl;
        cout << "4: " << s1.pop() << endl; //oops
    }
    catch(Stack::Full)
    {
        cout << "Exception: Stack Full" << endl;
    }
    catch(Stack::Empty)
    {
        cout << "Exception: Stack Empty" << endl;
    }
    return 0;
}

```

A group of catch blocks, or a catch ladder, operates a little like a switch statement, with only the appropriate section of code being executed.

1: 33
2: 22
3: 11
Exception: Stack Empty

The output

EXCEPTIONS WITH ARGUMENTS

```

class Distance //English Distance class
{
private:
    int feet;
    float inches;
public:
    //-----
    class InchesEx //exception class
    {
    public:
        string origin; //for name of routine
        float iValue; //for faulty inches value
        InchesEx(string or, float in) //2-arg constructor
        {
            origin = or; //store string
            iValue = in; //store inches
        }
    }; //end of exception class
    //-----
    Distance() //constructor (no args)
    { feet = 0; inches = 0.0; }
    //-----
    Distance(int ft, float in) //constructor (two args)
    {
        if(in >= 12.0)
            throw InchesEx("2-arg constructor", in);
        feet = ft;
        inches = in;
    }
    //-----
    void getdist() //get length from user
    {
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
        if(inches >= 12.0)
            throw InchesEx("getdist() function", inches);
    }
    //-----
    void showdist() //display distance
    { cout << feet << "'-" << inches << "'"; }
};

```

```

int main()
{
    try
    {
        Distance dist1(17, 3.5); //2-arg constructor
        Distance dist2; //no-arg constructor
        dist2.getdist(); //get value
        //display distances
        cout << "\ndist1 = "; dist1.showdist();
        cout << "\ndist2 = "; dist2.showdist();
    }
    catch(Distance::InchesEx ix) //exception handler
    {
        cout << "\nInitialization error in " << ix.origin
        << "\n Inches value of " << ix.iValue
        << " is too large.";
    }
    cout << endl;
    return 0;
}

```

Enter feet: 5
Enter inches: 2.7
dist1 = 17'-3.5"
dist2 = 5'-2.7"

Enter feet: 2
Enter inches: 13.4
Initialization error in getdist() function
Inches value of 13.4 is too large.

The outputs:

Specifying Data in an Exception Class: It's convenient to make the data in an exception class public so it can be accessed directly by the exception handler.

Initializing an Exception Object: In the two-argument constructor The string will tell us which member function is throwing the exception, and the value of inches will report the faulty inches value detected by the member function.

Extracting Data from the Exception Object:

THE BAD_ALLOC CLASS

13 Lecture 7

Standard C++ contains several built-in exception classes. The most commonly used is probably `bad_alloc`, which is thrown if an error occurs when attempting to allocate memory with `new`.

```
// badalloc.cpp
// demonstrates bad_alloc exception
#include <iostream>
using namespace std;
int main()
{
    const unsigned long SIZE = 10000; //memory size
    char* ptr; //pointer to memory
    try
    {
        ptr = new char[SIZE]; //allocate SIZE bytes
    }
    catch(bad_alloc) //exception handler
    {
        cout << "\nbad_alloc exception: can't allocate memory.\n";
        return(1);
    }
    delete[] ptr; //deallocate memory
    cout << "\nMemory use is successful.\n";
    return 0;
}
```

Put all the statements that use `new` in a `try` block. The `catch` block that follows handles the exception, often by displaying an error message and terminating the program.

The output:

Memory use is successful.
Press any key to continue

EXCEPTION NOTES

14 Lecture 7

Exceptions and Class Libraries:

An important problem solved by exceptions is that of errors in class libraries. A library routine may discover an error, but typically it doesn't know what to do about it.

- If you're writing a class library, you should cause it to throw exceptions for anything that could cause problems to the program using it.
- If you're writing a program that uses a class library, you should provide `try` and `catch` blocks for any exceptions that it throws.

Not for Every Situation:

Exceptions should not be used for every kind of error. E.g., exceptions should probably not be used for user input errors (such as inserting letters into numerical input) that are easily detectable by the program. Instead the program should use normal decisions and loops to check the user's input and ask the user to try again if necessary.

→ EXCEPTION NOTES

15

Lecture 7

Destructors Called Automatically:

The exception mechanism is surprisingly sophisticated. When an exception is thrown, a destructor is called automatically for any object that was created by the code up to that point in the try block. This is necessary because the application won't know which statement caused the exception, and if it wants to recover from the error, it will need to start over at the top of the try block.